



INSTITUTO DE ESTUDOS EUROPEUS SUPERIORES

Desenvolvimento de aplicação

Francisco Sampaio – 2023108756

(2023108756@cloud.iees.pt)

Ctsp- Tecnologias e Programação de Sistemas de Informação

Projeto de Tecnologia e Programação de Sistemas de Informação

Docente

Micael Soares

Índice

1	Planeamento do projeto	1
1.1	Introdução ao Software de Tracking	1
1.2	Organização do Projeto	1
2	Análise	3
2.1	Análise de viabilidade do projeto	3
2.2	Definição do objetivo	3
3	Desenvolvimento	4
3.1	Criação do cenário	4
3.1.1	ViewBox	4
3.1.2	Configuração Inicial do Cenário	5
3.1.3	Atualização do Cenário	6
3.1.4	Renderização do Cenário	6
3.2	Desenvolvimento do personagem	7
3.2.1	Definição e Configuração do Personagem	7
3.2.2	Física do Pássaro	8
3.2.3	Controle de Movimento (Flap)	8
3.2.4	Animação do Pássaro	8
3.2.5	Sincronização com o Jogo (game.py)	9
3.3	Criação de objetos	10
3.3.1	Criação da classe Pipe no pipe.py	10
3.4	Sistema de colisão	12

Lista de Figuras

1.1	Tabela de tarefas	2
1.2	Gráfico de Gantt	2

Capítulo 1

Planeamento do projeto

1.1 Introdução ao Software de Tracking

O ProjectLivre é uma ferramenta de gestão de projetos, amplamente utilizada por equipas e indivíduos para planear, organizar e monitorar o progresso de projetos. Ele combina funcionalidades de gerenciamento de tarefas, controle de prazos e colaboração entre membros, tudo em uma plataforma Open Source.

Neste projeto o ProjectLivre ajuda a manter o trabalho organizado e eficiente. Ele permite acompanhar as diferentes etapas do projeto e ajuda a garantir que os prazos são cumpridos. Além disso, por ser Open Source, é uma alternativa acessível e flexível, ideal para estudantes que precisam de uma ferramenta robusta sem custos adicionais. (Marques, 2019)

1.2 Organização do Projeto

Análise:

- Análise de viabilidade do projeto (1 dia);
- Definição do objetivo (1 dia);

Desenvolvimento:

- Criação do cenário (5 dias);
- Desenvolvimento do personagem (5 dias);
- Criação de objetos (5 dias);
- Sistema de colisão (5 dias);
- Score/Reset/Base de Dados (13 dias);

1		Analise	2 dias?	03-11-2024 8:00	05-11-2024 17:00
2		Analise de viabilidade do pr	1 dia?	03-11-2024 8:00	04-11-2024 17:00
3		Defenição do Objetivo	1 dia?	05-11-2024 8:00	05-11-2024 17:00
4		Desenvolvimento	31 dias?	05-11-2024 8:00	17-12-2024 17:00
5		Criação de Cenário	5 dias	05-11-2024 8:00	11-11-2024 17:00
6		Desenvolvimento do Person	5 dias	12-11-2024 8:00	18-11-2024 17:00
7		Criação de Objetos	5 dias	19-11-2024 8:00	25-11-2024 17:00
8		Sistema de Colisão	4 dias	26-11-2024 8:00	29-11-2024 17:00
9		Score/Reset/Base de dado	13 dias?	29-11-2024 8:00	17-12-2024 17:00

Figura 1.1: Tabela de tarefas

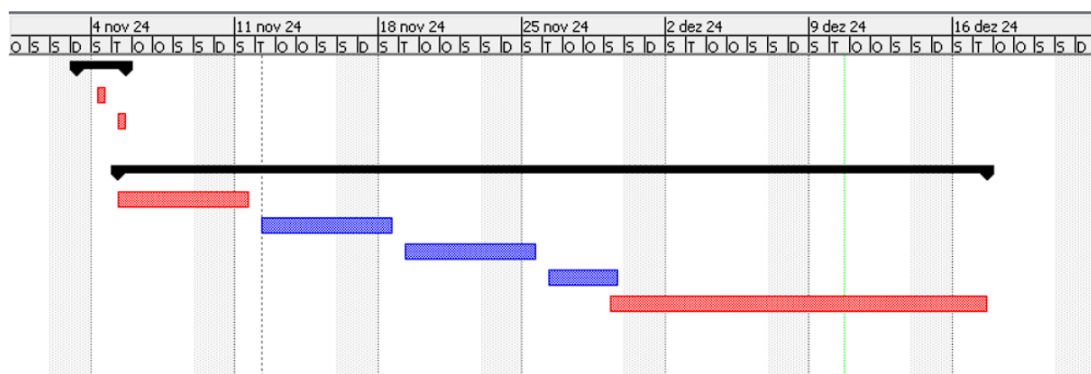


Figura 1.2: Gráfico de Gantt

Capítulo 2

Análise

2.1 Análise de viabilidade do projeto

Para o meu projeto de desenvolvimento de jogos, escolhi recriar o clássico Flappy Bird no Visual Studio Code (VSCode). A decisão foi motivada pela simplicidade do jogo, que é um exemplo típico de um jogo em 2D e, ao mesmo tempo, possui mecânicas básicas, que são ideais para explorar e aplicar conceitos fundamentais de programação e design de jogos.

Outro fator determinante para a escolha foi a disponibilidade de recursos e bibliotecas de assets online. Para desenvolver o jogo, utilizei a linguagem Python ligado á biblioteca Pygame, que é usada para a criar jogos em 2D. A Pygame provê um conjunto de ferramentas para manipulação de gráficos, sons e eventos, que facilita o desenvolvimento de aspetos como o movimento do personagem, colisões e animações.

Os recursos gráficos, como imagens, foram obtidos em repositórios online, como o **FlappyBird-Master**. Estes materiais foram fundamentais para o projeto e para criar uma experiência visual agradável para o usuário. Com este projeto, além de reforçar conceitos de programação em Python, aprofundei o meu conhecimento sobre bibliotecas específicas para jogos e práticas de integração de assets, o que é uma etapa essencial no desenvolvimento de jogos independentes.

2.2 Definição do objetivo

O objetivo deste projeto é recriar o Flappy Bird em Python com o uso da biblioteca Pygame, com o intuito de explorar e aplicar conceitos de desenvolvimento de jogos em 2D. Pretendo compreender as etapas envolvidas no processo de criação de um jogo simples, incluindo a implementação de mecânicas básicas, gerenciamento de eventos, manipulação de gráficos e integração de assets visuais e sonoros. Adicionalmente, o projeto tem como objetivo desenvolver as minhas habilidades práticas em programação na criação de jogos e incentivar a utilização de recursos disponíveis online para otimizar o processo de desenvolvimento.

Por fim, o trabalho visa proporcionar uma experiência interativa e funcional que simule, de forma fiel, as características do jogo original.

Capítulo 3

Desenvolvimento

3.1 Criação do cenário

3.1.1 ViewBox

A ViewBox no Pygame é uma área que define a região visível de um elemento gráfico. Ela serve para ajustar, redimensionar ou posicionar o conteúdo desenhado dentro de uma tela ou janela.

Listing 3.1: Carregamento das imagens

```
1      import pygame as pg
2  import sys
3  pg.init()
4
5  class Game:
6      def __init__(self):
7          self.width = 600
8          self.height = 768
9          self.win = pg.display.set_mode((self.width, self.height))
10         self.gameLoop()
11
12     def gameLoop(self):
13         while True:
14             for event in pg.event.get():
15                 if event.type == pg.QUIT:
16                     pg.quit()
17                     sys.exit()
18
19 game = Game()
```

3.1.2 Configuração Inicial do Cenário

Na função "SetUpBgAndGround", são carregadas as imagens do fundo e do chão, que serão exibidas durante o jogo. A configuração do cenário é feita com as seguintes etapas:

- **Carregamento das Imagens:** As imagens do fundo e do chão são carregadas a partir de arquivos armazenados na pasta assets. Elas são ajustadas ao tamanho desejado com o método `pg.transform.scale_by`, utilizando o fator de escala definido na variável `scale_factor`.

```
1         self.bg_img=pg.transform.scale_by(pg.image.load("assets/bg.png").
2             convert(),self.scale_factor)
3         self.ground1_img=pg.transform.scale_by(pg.image.load("assets/ground
4             .png").convert(),self.scale_factor)
5         self.ground2_img=pg.transform.scale_by(pg.image.load("assets/ground
6             .png").convert(),self.scale_factor)
```

- **Posicionamento do Chão:** Duas imagens do chão são usadas para criar um efeito contínuo enquanto o jogador avança. Os retângulos (Rect) que representam essas imagens são posicionados lado a lado. O primeiro começa na posição `x = 0` e o segundo logo após o final do primeiro.

```
1         self.ground1_rect=self.ground1_img.get_rect()
2         self.ground2_rect=self.ground1_img.get_rect()
3
4         self.ground1_rect.x=0
5         self.ground2_rect.x=self.ground1_rect.right
6         self.ground1_rect.y=568
7         self.ground2_rect.y=568
```


3.1.3 Atualização do Cenário

O cenário precisa ser atualizado constantemente para dar a sensação de movimento. Isso ocorre dentro da função `UpdateEverything`.

- **Movimento do Chão:** O chão é deslocado horizontalmente para a esquerda, simulando o movimento do jogo. Quando uma das imagens do chão sai completamente da tela (`right < 0`), ela é reposicionada para o final da outra, criando um efeito de loop.

```

1         if self.is_enter_pressed:
2             self.ground1_rect.x-=self.move_speed*dt
3             self.ground2_rect.x-=self.move_speed*dt
4
5             if self.ground1_rect.right<0:
6                 self.ground1_rect.x=self.ground2_rect.right
7             if self.ground2_rect.right<0:
8                 self.ground2_rect.x=self.ground1_rect.right

```

- **Velocidade de Movimento:** O valor `self_move_speed` controla a velocidade de movimento do cenário . Ele é multiplicado pelo `dt` (delta time), que representa o tempo entre frames, garantindo que o movimento seja uniforme, independentemente da taxa de frames.

3.1.4 Renderização do Cenário

A exibição do cenário é feita na função `DrawEverything`. O fundo é desenhado primeiro, seguido pelas duas imagens do chão. Essa ordem garante que o chão sobreponha o fundo.

```

1         self.win.blit(self.bg_img, (0, -300))
2         self.win.blit(self.ground1_img, self.ground1_rect)
3         self.win.blit(self.ground2_img, self.ground2_rect)

```

- A posição do fundo é fixa, enquanto o chão utiliza os retângulos (`self_ground1_rect` e `self_ground2_rect`) para definir sua posição na tela.

3.2 Desenvolvimento do personagem

O desenvolvimento do personagem, o pássaro, no jogo, é possível através da classe Bird definida no arquivo bird.py. Essa classe é responsável por gerir as propriedades, comportamentos e animações do pássaro, além de interagir com a física do jogo.

3.2.1 Definição e Configuração do Personagem

No arquivo bird.py, a classe Bird é responsável por criar e gerir o pássaro no jogo. Quando a classe é usada, o método `__init__` define várias características do pássaro, como a sua aparência, posição e comportamento físico.

Dentro de `__init__`, a primeira etapa é carregar as imagens que representarão o pássaro nas diferentes posições de animação. No caso, o pássaro tem duas imagens: uma para a posição de "subida" e outra para a posição de "descida", que formam a animação do movimento do pássaro durante o jogo.

```
1         self.img_list=[pg.transform.scale_by( pg.image.load("assets/birdup.  
           png").convert_alpha(),scale_factor),  
2                 pg.transform.scale_by( pg.image.load("assets/  
           birddown.png").convert_alpha(),scale_factor)]
```

Essas imagens são carregadas e ajustadas ao tamanho do jogo através do `scale_factor`. A lista `self.img_list` contém essas duas imagens, e a variável `self.image_index` controla qual imagem está a ser exibida de momento, inicialmente configurada para 0 (a primeira imagem, "subindo").

Depois, é configurado o retângulo (`self.rect`) que representa a posição do pássaro na tela. O centro do retângulo é inicialmente posicionado em (100, 100), que é a posição inicial do pássaro.

```
1 self.rect = self.image.get_rect(center=(100, 100))
```

Além disso, são definidas as propriedades de física do pássaro, como a velocidade em y (`self.y_velocity`), a gravidade (`self.gravity`), a velocidade do flap (batida das asas, representado por `self.flap_speed`), e o contador de animação (`self.anim_counter`), que controla a troca das imagens de animação.

3.2.2 Física do Pássaro

Para dar a sensação de movimento realista ao pássaro, a física é crucial. O pássaro tem um comportamento que imita a gravidade: ele vai caindo ao longo do tempo, a menos que o jogador interaja com o jogo (com o espaço, para fazer o pássaro bater as asas).

Através de "applyGravity", a gravidade é aplicada ao pássaro em cada frame. A velocidade em y do pássaro é implementada pela gravidade multiplicada pelo tempo (dt), o que faz com que o pássaro caia cada vez mais rápido conforme o jogo avança.

```
1 def applyGravity(self, dt):
2     self.y_velocity += self.gravity * dt
3     self.rect.y += self.y_velocity
```

Essa atualização faz com que a posição do pássaro mude verticalmente, com a gravidade puxando-o para baixo. O valor de dt garante que a física seja independente da taxa de quadros, mantendo o movimento consistente em diferentes dispositivos.

3.2.3 Controle de Movimento (Flap)

O movimento do pássaro é controlado pelas batidas das asas, que é ativada quando o jogador pressiona a tecla de espaço. Através do flap, a velocidade do pássaro em y é definida como um valor negativo, o que faz o pássaro "subir" ao ser "chamado".

```
1 def flap(self, dt):
2     self.y_velocity = -self.flap_speed * dt
```

A variável flap_speed controla a intensidade desse movimento de subida, e a operação -self.flap_speed * dt assegura que o movimento seja suave e proporcional ao tempo de cada frame.

3.2.4 Animação do Pássaro

O pássaro tem uma animação, alternando entre as imagens de "subida" e "descida" para simular o movimento das asas, através do playAnimation, que troca a imagem do pássaro a cada certo número de frames (self.anim_counter). A cada 5 frames, a imagem do pássaro é trocada entre "subindo" e "descendo", criando o efeito de batida das asas.

```
1 def playAnimation(self):
2     if self.anim_counter == 5:
3         self.image = self.img_list[self.image_index]
4         if self.image_index == 0:
5             self.image_index = 1
6         else:
7             self.image_index = 0
8         self.anim_counter = 0
9     self.anim_counter += 1
```

3.2.5 Sincronização com o Jogo (game.py)

No arquivo game.py, o pássaro é gerido dentro da classe Game. Quando iniciamos o jogo, o pássaro é criado com um fator de escala que o ajusta ao tamanho da tela.

```
1 self.bird = Bird(self.scale_factor)
```

Dentro do loop principal do jogo (gameLoop), o pássaro é atualizado a cada frame. A função update do pássaro é chamada para aplicar a gravidade e a animação, além de verificar se o pássaro atingiu o topo da tela ou o chão, ajustando a sua posição conforme necessário.

```
1 self.bird.update(dt)
```

Além disso, a interação do jogador com o pássaro é feita através da captura de eventos do teclado. Quando a tecla SPACE é pressionada, o método flap do pássaro é chamado, fazendo com que ele suba.

```
1 if event.type == pg.KEYDOWN:  
2     if event.key == pg.K_SPACE and self.is_enter_pressed:  
3         self.bird.flap(dt)
```

Isto permite que o jogador controle a ascensão do pássaro.

3.3 Criação de objetos

Nesta etapa, será explicado como os objetos são criados e manipulados no jogo, com foco nas classes Pipe (no arquivo pipe.py) e Game (no arquivo Game.py). Serão descritos os processos de arranque, atualização, desenho e remoção dos objetos, destacando a sincronização entre os dois arquivos.

3.3.1 Criação da classe Pipe no pipe.py

No arquivo pipe.py, criamos a classe Pipe, responsável pela criação e movimento dos tubos que servem como obstáculos no jogo. Esta classe utiliza a biblioteca pygame para manipular imagens e coordenadas no ecrã. Quando um objeto da classe Pipe é chamado, o método `__init__`, carrega as imagens dos tubos usando o `pg.image.load` e redimensiona-las através do `pg.transform.scale_by`. As imagens utilizadas são `pipeup.png` para o tubo virado para cima e `pipedown.png` para o tubo virado para baixo, ambas na pasta `assets`. O redimensionamento é feito com base no parâmetro `scale_factor`.

```
1     self.img_up = pg.transform.scale_by(pg.image.load("assets/pipeup.png").
        convert_alpha(), scale_factor)
2 self.img_down = pg.transform.scale_by(pg.image.load("assets/pipedown.png").
        convert_alpha(), scale_factor)
```

De seguida, são criados dois retângulos associados às imagens com o método `get_rect()` fornecido pela biblioteca pygame. Estes retângulos, chamados `rect_up` e `rect_down`, são fundamentais para definir as posições dos tubos no ecrã e também para as colisões no jogo.

```
1     self.rect_up = self.img_up.get_rect()
2 self.rect_down = self.img_down.get_rect()
```

Depois de criar os retângulos, é definida a posição vertical do tubo superior, chamada `rect_up.y`, utilizando a função `randint` da biblioteca `random`. Esta função gera um valor aleatório entre 250 e 520 que permite com que os tubos apareçam em alturas diferentes a cada nova criação. A posição do tubo inferior (`rect_down.y`) é calculada com base na altura do tubo superior e na distância entre os tubos, definida pelo atributo `self.pipe_distance`. O valor de `pipe_distance` é fixado em 130, o que garante um espaço constante entre os tubos.

```
1     self.rect_up.y = randint(250, 520)
2 self.rect_down.y = self.rect_up.y - self.pipe_distance - self.rect_up.
    height
```

A posição horizontal inicial dos tubos é definida fora do ecrã, mais precisamente na coordenada $x = 600$, o que garante que os tubos entrem no ecrã pela direita à medida que se movem em direção à esquerda. A velocidade de movimento é controlada através do `move_speed`, que também é definido no momento da criação do objeto.

```
1     self.rect_up.x = 600
2 self.rect_down.x = 600
```

Assim, a classe `Pipe` cria dois tubos com posições definidas aleatoriamente para o tubo superior e ajustadas para o tubo inferior, garantindo um espaço entre eles. Estes tubos começam fora do ecrã e se movem em direção à esquerda conforme o jogo prossegue, funcionando como obstáculos dinâmicos no cenário do jogo

3.4 Sistema de colisão

O sistema de colisão é responsável por verificar se o pássaro entra em contacto com os tubos ou com o chão. Esta verificação é feita dentro do método `CheckCollisions`, que faz parte da classe `Game`. O sistema utiliza as funcionalidades da biblioteca `pygame`, como os retângulos de colisão associados aos objetos, para determinar se ocorreu uma colisão. Quando isso acontece, as ações necessárias são tomadas para interromper o jogo.

O método `checkCollisions` começa por verificar se a lista `self.pipes` contém algum tubo, garantindo que não ocorram erros caso a lista esteja vazia. Se houver pelo menos um tubo, o sistema de colisão prossegue com as verificações necessárias.

```
1  if len(self.pipes):
```

A primeira verificação é feita para determinar se o pássaro bateu no chão. A coordenada vertical `bottom` do retângulo do pássaro (`self.bird.rect.bottom`) é comparada com a posição `y = 568`, que corresponde à posição do chão no jogo. Caso o pássaro toque no chão, o atributo `update_on` da classe `Bird` é definido como `False`, e o atributo `is_enter_pressed` é também definido como `False`, o que faz com que o pássaro deixe de se mover e o jogo seja interrompido.

```
1  if self.bird.rect.bottom > 568:
2  self.bird.update_on = False
3  self.is_enter_pressed = False
```

Depois, é verificada a colisão do pássaro com os tubos. As colisões são verificadas utilizando o método `colliderect`, que é fornecido pela biblioteca `pygame`. Este método compara o retângulo do pássaro (`self.bird.rect`) com os retângulos dos tubos superior (`self.pipes[0].rect_up`) e inferior (`self.pipes[0].rect_down`). Caso haja sobreposição entre os retângulos, é considerada uma colisão e o jogo é interrompido ao definir o atributo `is_enter_pressed` como `False`.

```
1  if (self.bird.rect.colliderect(self.pipes[0].rect_down) or
2  self.bird.rect.colliderect(self.pipes[0].rect_up)):
3  self.is_enter_pressed = False
```

Dessa forma, o sistema de colisão é implementado no `Game.py` para garantir que o jogo reconheça quando o pássaro colide com os tubos ou toca no chão. O método `checkCollisions` é chamado repetidamente dentro do `gameLoop` durante a execução do jogo, garantindo que as verificações de colisão ocorram em tempo real. Quando uma colisão é detectada, o movimento do pássaro é interrompido e o jogador perde a possibilidade de continuar o jogo até ser reiniciar.